

QueueFlow: Computer Vision Based Automated Queue Ticket Assignment and Real-Time Predictive System for Dynamic Campus Service Delivery

A Capstone Research

Presented to the Faculty of the
College of Computer Studies
Naga College Foundation, Inc.
Naga City

In Partial Fulfillment of
The Requirements for the Degree
Bachelor of Science in Computer Science

Balbin, Archie D.
Sanota, Joshua Rovic P.

NAGA COLLEGE FOUNDATION, INC.
College of Computer Studies
City of Naga

APPROVAL SHEET

This capstone research of **ARCHIE D. BALBIN** and **JOSHUA ROVIC P. SAN-OTA** entitled “**QUEUEFLOW: COMPUTER VISION BASED AUTOMATED QUEUE TICKET ASSIGNMENT AND REAL-TIME PREDICTIVE SYSTEM FOR DYNAMIC CAMPUS SERVICE DELIVERY**” was prepared and submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science. It has been examined and is recommended for acceptance and approval for oral examination.

Fritzgerald Enric T. Imperial, MIT
Adviser

THESIS COMMITTEE:

[CHAIRMAN NAME]
Chairman

[PANEL MEMBER 1]
Panel Member

[PANEL MEMBER 2]
Panel Member

[SECRETARY NAME]
Panel Secretary

PANEL OF EXAMINERS

Approved by the Committee on Oral Examinations on _____.

[CHAIRMAN NAME]
Chairman

[PANEL MEMBER 1]
Panel Member

[PANEL MEMBER 2]
Panel Member

Accepted and approved in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science.

ANNA LORETTA C. ROMULO, MSIT
Dean

NAGA COLLEGE FOUNDATION, INC.
College of Computer Studies
City of Naga

SECRETARY'S CERTIFICATION

This capstone research of **ARCHIE D. BALBIN** and **JOSHUA ROVIC P. SANTOTA** entitled **“QUEUEFLOW: COMPUTER VISION BASED AUTOMATED QUEUE TICKET ASSIGNMENT AND REAL-TIME PREDICTIVE SYSTEM FOR DYNAMIC CAMPUS SERVICE DELIVERY”** was prepared and submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science. It has been examined and is recommended for acceptance and approval for oral examination.

[**SECRETARY NAME**]
Secretary

NAGA COLLEGE FOUNDATION, INC.
College of Computer Studies
City of Naga

EDITOR'S CERTIFICATION

This capstone research of **ARCHIE D. BALBIN** and **JOSHUA ROVIC P. SANTOTA** entitled **“QUEUEFLOW: COMPUTER VISION BASED AUTOMATED QUEUE TICKET ASSIGNMENT AND REAL-TIME PREDICTIVE SYSTEM FOR DYNAMIC CAMPUS SERVICE DELIVERY”** was prepared and submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science. It has been examined and is recommended for acceptance and approval for oral examination.

[EDITOR NAME]
Editor

ACKNOWLEDGMENT

The researchers would like to extend their thanks to ...

Abstract

Title:	QueueFlow: Computer Vision Based Automated Queue Ticket Assignment and Real-Time Predictive System for Dynamic Campus Service Delivery
Researcher:	Archie D. Balbin and Joshua Rovic P. Sanota
Type of Document:	Unpublished Undergraduate Capstone Research (2027)
Host/Accrediting School:	Naga College Foundation, Inc.
City:	Naga City
Region:	Region V
Year of Completion:	2027
Adviser:	Fritzgerald Enric T. Imperial, MIT
Keywords:	Queue management, computer vision, YOLOv8, person re-identification, predictive analytics, FastAPI, printer-agnostic ticket protocol, QR code, JWT, M/M/c model, Holt's double exponential smoothing

Queueing in many campus service offices is still handled by staff observation, manual numbering, and verbal updates at the counter. During enrollment, payment, and clearance periods, this makes it difficult for students to know their turn and for staff to judge how quickly the line is changing. QueueFlow was developed as a capstone prototype for this setting. It combines a YOLOv8n detector, OpenCV camera processing, automatic queue-number assignment, ticket issuance, and short-horizon wait-time forecasting. The prototype runs the camera detector separately from the FastAPI backend: the detector sends tracked-person metadata, duplicate-filtered detections, crowd counts, and annotated snapshots, while the backend confirms entrants inside a calibrated queue zone before issuing a number. Confirmation uses a candidate window, motion checks, track-ID remapping, and a hybrid appearance-spatial re-entry rule so that brief exits from the camera view do not immediately produce duplicate queue tickets. QueueFlow also uses a printer-agnostic ticket payload containing the queue number, QR link, short code, and JSON Web Token (JWT), which can be rendered as a thermal ticket in deployment or as a PDF ticket in the prototype build. Queue records are stored in MySQL, students can check ticket status from a mobile client, and staff can monitor live snapshots, no-show alerts, counter actions, analytics, and forecasts through the dashboard. The forecasting component reports the current estimated wait time and 5-, 15-, and 30-minute projections using an M/M/c baseline, local trend projection, Holt's double exponential smoothing, and a mean-reversion adjustment. The study evaluates the Naga College Foundation, Inc. (NCF) prototype through queue assignment accuracy, re-identification reliability, ticket and token validation, prediction error using MAE and MAPE, system latency, and

user satisfaction.

Contents

1	Introduction	13
1.1	Background of the Study	13
1.2	Review of Related Studies	14
1.2.1	Computer Vision for Real-Time Person Detection	14
1.2.2	YOLO-Based Tracking and Deduplication	15
1.2.3	Queue Management and Person Re-Identification	15
1.2.4	Predictive Analytics for Service Demand Forecasting	16
1.3	Synthesis of the State-of-the-Art	16
1.4	Gap Bridged by the Study	18
1.5	Theoretical Framework	18
1.6	Conceptual Framework	20
1.7	Statement of the Problem	22
1.8	Assumptions	22
1.9	Scope and Delimitation	23
1.10	Significance of the Study	23
1.11	Definition of Terms	24
2	Methodology	26
2.1	Research Design	26
2.2	Population and Locale of the Study	26
2.3	Sample and Sampling Technique	27
2.4	Data Gathering Procedure	27
2.5	Research Instrument	28
2.6	Ethical Considerations	29
2.7	Statistical Treatment of Data	31
2.8	System Design and Architecture	32
2.8.1	Design Principles	32
2.8.2	Tools and Technologies	34
2.8.3	System Architecture	35
2.8.4	Machine-Learning Wait-Time Predictor Training Protocol	38
2.9	Software Development Process	40
2.10	Validation and Testing Methods	42
3	Results and Discussions	44
3.1	Results	44
3.2	Design and System Plans	44
3.3	Summary of Findings	44
3.4	Conclusions	44
3.5	Recommendations	44

	10
A Synthesis Matrix of Related Literature	45
B QueueFlow Evaluation Questionnaire	47
Curriculum Vitae	57
Appendices	58

List of Figures

1.1	Theoretical Framework of QueueFlow	19
1.2	Conceptual Framework of QueueFlow	21
2.1	QueueFlow Six-Layer System Architecture	35
2.2	QueueFlow Detection-to-Ticket Pipeline	37
2.3	Agile Development Process for QueueFlow. Each of the eight sprints enumerated above completes one revolution of this five-phase cycle—Plan, Design, Build, Test, Review—before the next sprint begins. The Review phase routes adviser feedback and measured performance back into the Plan phase of the following sprint, instantiating, at the development level, the feedback loop described in Section 1.6.	42

List of Tables

2.1	Likert Scale Interpretation	29
2.2	System Tools and Technologies	34
A.1	Synthesis Matrix of Related Literature	46
B.1	Likert Scale Interpretation	48
B.2	Dimension A: Usability	49
B.3	Dimension B: Detection Reliability	50
B.4	Dimension C: Prediction Usefulness	51
B.5	Dimension D: Dashboard Clarity	52
B.6	Dimension E: Overall Satisfaction	53

1 Introduction

1.1 Background of the Study

In a busy campus office, the queue is not only a line of students; it is also an early signal of how much pressure the office is about to handle. During enrollment, payment, clearance, and related transactions, students may arrive faster than staff can manually count, organize, and update them. The result is a familiar but difficult situation: students wait without a clear view of their status, while staff make service, skip, recall, and counter decisions with only partial real-time information.

This problem is not unique to Naga College Foundation, Inc. (NCF). Recent reports from Philippine higher education institutions document the persistence of long enrollment queues, congested service windows, and the operational gap between rising student volumes and manual queue management capacity [1]. At NCF, informal accounts from students and service staff indicate that wait times during enrollment, payment, and clearance periods can become substantial, with congestion most pronounced during the first weeks of each semester. This pattern, combined with the absence of a real-time queue display and the manual recording of transaction order, indicates the same kind of inefficient service delivery that the literature identifies as a recurring issue in Philippine HEIs.

At NCF, congestion also affects the continuity of student transactions. Enrollment, cashiering, and library services are separate offices, but students often experience them as one connected process. A delay at one window can affect the next step that the student needs to complete. The Commission on Higher Education recognizes accessibility and efficient service delivery as indicators of institutional quality [2]; for this reason, improving queue visibility is treated in this study as part of improving the student service experience, not merely as a counter-management task.

Conventional queue systems are useful once a student has requested a number or once staff have encoded the transaction. Their limitation is that they normally begin after the office has already recognized the arrival. They can store issued tickets, but they do not usually observe the service area, confirm new arrivals from a camera view, reduce duplicate queue entries, or estimate near-term changes in the line. The gap is therefore not only ticket issuance; it is the missing link among arrival detection, staff monitoring, student status access, and prediction.

QueueFlow responds to this gap through a camera-connected queue management prototype for campus service offices. Instead of waiting for a manual entry, it uses YOLOv8n and OpenCV to detect persons inside a defined queue zone. YOLO is appropriate for this use case because it performs object detection in a single network pass [3]. YOLOv8n was selected because it is light enough for prototype-grade hardware while still supporting persistent tracking and stable deployment tooling [4, 5]. Newer YOLO variants, including YOLOv11 and YOLOv12, introduce architectural improvements [6, 7]; however, this study prioritizes stable single-class person detection in a controlled

queue zone over marginal accuracy gains that may require more computing resources. The selection therefore reflects the constraints of the prototype environment rather than a general claim that YOLOv8n is superior in all settings.

QueueFlow extends detection with duplicate filtering, candidate confirmation, and hybrid re-entry matching so that queue records remain more consistent when people move or briefly leave the camera frame. The queue ticket component is implemented as a printer-agnostic ticket protocol: each confirmed queue entry produces a queue number, a short code, a QR link, and a JSON Web Token (JWT) that together allow the student to verify the ticket and check status. The intended deployment renders this ticket on thermal-printed paper at the service counter; in the prototype demonstration, the same ticket payload is rendered as a downloadable PDF. The research contribution of QueueFlow is in the ticket protocol and its integration with detection and prediction, not in the physical printing mechanism, which is a deployment-layer concern.

Beyond ticket generation, QueueFlow connects the detector, queue records, staff dashboard, and prediction module through a FastAPI-based backend and MySQL database. The prediction module supports current, 5-minute, 15-minute, and 30-minute wait-time estimates using queuing logic and short-term forecasting. Through this design, the study aims to provide NCF with a prototype that improves queue visibility, supports staff decision-making, and demonstrates how computer vision and predictive analytics can be applied to campus service management in a Philippine educational institution.

The Review of Related Literature therefore examines the technologies and concepts that support each part of the problem identified above. Computer vision and YOLO literature address automated queue observation, queue management literature explains ticketing and identity maintenance, and forecasting literature supports the wait-time estimates needed for staff decision-making.

1.2 Review of Related Studies

1.2.1 Computer Vision for Real-Time Person Detection

Computer vision has matured significantly through deep convolutional neural networks, enabling non-intrusive observation of crowd density and movement in public service areas. Sindagi and Patel [8] surveyed CNN-based approaches to single-image crowd counting and density estimation, documenting how deep learning has transformed earlier methods limited to small variations in scale and density into systems capable of handling occlusions, complex scenes, and dense crowds. One-stage detectors such as YOLO have enabled live deployment at frame rates suitable for real-time monitoring [3]. A comprehensive review by Terven et al. [9] traces YOLO’s evolution from YOLOv1 through YOLOv8 and YOLO-NAS, examining the architectural innovations that drive each iteration’s accuracy-speed tradeoff. Wang et al. [5] more recently proposed a lightweight YOLOv8n-derived architecture for accurate person detection on edge-device surveillance, confirming the suitability of this model class for resource-constrained deployments and reinforcing the practical viability of YOLOv8n as a base for person-detection prototypes.

Privacy concerns are an essential consideration in any camera-based system. Padilla-

Lopez et al. [10] survey visual privacy protection methods, categorizing approaches such as anonymization at the camera level, selective scrambling, and aggregate-only transmission. These principles inform QueueFlow’s design choice to process bounding-box data and colour histograms without storing or transmitting raw video.

1.2.2 YOLO-Based Tracking and Deduplication

YOLO’s single-pass detection architecture delivers the speed necessary for real-time queue monitoring, processing entire frames in one forward pass to simultaneously predict bounding boxes, confidence scores, and class labels [3]. The YOLOv8 series advances this through improved multi-scale feature pyramid networks and anchor-free detection heads [4], with the broader YOLO family reviewed in depth by Terven et al. [9]. More recent variants extend this trajectory in different directions: YOLOv11 emphasizes a reduced parameter count and improved feature reuse, while YOLOv12 introduces an attention-centric architecture with Area Attention and Residual ELAN modules [6, 7]. These newer variants improve accuracy and contextual reasoning but at marginal latency cost; for single-class person detection in a fixed queue zone, the stability, tracking integration, and benchmark maturity of YOLOv8n remain practically advantageous.

Beyond single-frame detection, multi-object tracking requires associating detections across frames into consistent identities. Wojke et al. [11] introduced the DeepSORT algorithm, which integrates appearance information with motion-based tracking via Kalman filtering and the Hungarian algorithm, reducing identity switches when targets are temporarily occluded. QueueFlow draws on similar principles in its custom track-ID remapping logic, while adding a domain-specific NMS pre-filter using X-column overlap and Y-adjacency thresholds to suppress the duplicate head/body boxes YOLO commonly emits for persons at close camera range.

1.2.3 Queue Management and Person Re-Identification

Camera-based monitoring of service areas enables passive observation of arrivals without requiring user interaction. The core technical challenge beyond detection is person re-identification: when a tracked individual temporarily disappears from view and reappears, the system must recognize them as the same person and restore their queue position rather than assigning a new number.

Ye et al. [12] provide a comprehensive survey of deep learning approaches to person re-identification, dividing methods into closed-world and open-world settings and categorizing the field along three axes: deep feature representation learning, deep metric learning, and ranking optimization. Building on this foundation, more recent reviews by Wang et al. [13] and Asperti et al. [14] document a shift toward Transformer-based representations, self-supervised and unsupervised methods, and improved robustness to occlusion and pose variation. These analyses underscore that practical Re-ID systems typically combine multiple signals: appearance-based methods using colour histograms or deep feature embeddings, spatial-temporal methods using last known position and elapsed time, and hybrid methods combining both. QueueFlow implements a hybrid re-entry matching approach in this tradition: it first matches a new detection to a missing

person by spatial proximity, then by appearance histogram similarity, and finally by recency as a last resort. This simplified scheme is appropriate for a single-zone prototype and avoids the additional computational overhead of deep Re-ID embeddings, which are typically necessary only when matching across non-overlapping cameras.

At the application layer, Concepcion et al. [1] describe a Smart Queuing and Appointment Management System for Bulacan State University, demonstrating that web-based automated queue management and online scheduling in a Philippine HEI can reduce wait times and improve perceived service quality. That system, however, operates through user-initiated web ticketing and appointment booking rather than passive computer-vision detection of arrivals, which is the gap QueueFlow specifically targets.

1.2.4 Predictive Analytics for Service Demand Forecasting

Multi-horizon forecasting provides both immediate and strategic insights by generating predictions at different time scales, each matched to a specific operational decision type. Shmueli and Koppius [15] formalize predictive analytics as a research paradigm distinct from explanatory modelling, emphasizing the role of empirical methods that generate and assess data predictions for decision support.

The M/M/c queuing model provides analytical wait-time calculations from real-time arrival and service rate measurements, making it suitable for current wait-time estimates and near-term tactical predictions [16]. Together with Little’s Law, the M/M/c framework gives QueueFlow the mathematical basis for translating live arrival rates into expected waiting times. System utilization is expressed as:

$$\rho = \frac{\lambda}{c\mu} \quad (1.1)$$

where λ is the arrival rate, μ the service rate, and c the number of active servers. The mean number in the system together with Little’s Law yields the expected wait time used by the dashboard:

$$W = \frac{L}{\lambda} \quad (1.2)$$

For medium-term forecasting, Holt’s double exponential smoothing tracks both queue level and trend with separate smoothing parameters [17]. Hyndman and Athanasopoulos [18] provide a comprehensive textbook treatment of exponential smoothing methods and multi-horizon forecasting, including discussion of damping factors that prevent over-projection during transient spikes—a practical necessity in campus service environments where queue lengths can surge and dissipate quickly. For the 30 min horizon implemented in QueueFlow, mean-reversion logic is used to limit long-forecast uncertainty.

1.3 Synthesis of the State-of-the-Art

The literature reviewed in this chapter supports the central concern identified in the Background of the Study: campus service offices need better real-time visibility of queue conditions, not merely a list of issued numbers. Studies on computer vision show

that camera-based person detection can reduce dependence on manual observation by capturing activity in a service area as it happens. YOLO-based approaches are especially relevant because they provide the speed needed for live monitoring, making them suitable for a prototype that must observe students entering and moving through a defined queue zone.

Within the YOLO family, the trajectory from YOLOv8 through YOLOv11 and the attention-centric YOLOv12 [7] indicates clear architectural progress, but also a widening gap between research benchmarks and the operational constraints of campus prototypes. YOLOv8n was selected for QueueFlow because it offers (a) stable production tooling and Ultralytics-supported persistent tracking, (b) lightweight inference suitable for the available prototype hardware, (c) broad community benchmarking that simplifies reproducibility, and (d) accuracy that is competitive with newer variants for the specific task of single-class person detection in a fixed queue zone. This rationale, rather than version recency, justifies the design choice.

However, the reviewed studies also show that detection is only the first layer of a complete queue management system. A person detector can identify bounding boxes, but it does not automatically solve the operational problems described in the Background: duplicate entries, temporary disappearance from the camera view, skipped queue positions, or student access to queue status. This makes tracking, deduplication, candidate confirmation, and re-entry matching necessary additions when computer vision is applied to actual service-office queues.

Research on queue management and predictive analytics further reinforces the need to move beyond static ticket numbering. Traditional queue tools can arrange students in order, but they provide limited support for staff decisions when demand changes quickly. Queuing theory and short-term forecasting address this limitation by translating arrival rates, active counters, and queue length into wait-time estimates. Multi-horizon prediction is therefore useful because staff need both immediate queue status and near-future indicators for counter planning.

Taken together, the reviewed works suggest that an effective campus queue solution should combine four functions in one workflow: camera-based queue entry, reliable queue identity management, ticket and status access for students, and predictive information for staff. These findings connect directly to QueueFlow’s design, which integrates YOLOv8n detection, hybrid re-entry matching, a printer-agnostic ticket protocol rendered as a thermal ticket in deployment and as a PDF in the demonstration build, a staff dashboard, and current, 5-minute, 15-minute, and 30-minute wait-time estimates.

Despite these advances, existing studies remain limited when viewed against the specific needs of NCF’s campus service context. Many focus on crowd counting, general queue applications, or prediction models as separate components. The Philippine HEI work that does exist, such as Concepcion et al. [1], addresses web-based ticketing and appointment-booking systems rather than passive vision-based detection. Fewer works demonstrate how these components can operate together in a Philippine educational institution where queue visibility, staff action, student status checking, and practical deployment constraints must be handled in a single prototype.

1.4 Gap Bridged by the Study

The literature reviewed in Section 1.2 shows strong work on individual parts of the problem: queue monitoring, object detection, re-identification, and wait-time forecasting. What remains less developed is a campus-service prototype that joins these parts into one operating workflow for a Philippine higher education institution. Existing Philippine HEI work, including the system reported by Concepcion et al. [1], is centered on web-based ticketing and appointment booking. It does not begin with passive camera-based recognition of arrivals in the service area.

QueueFlow narrows this gap by connecting the arrival, identity, ticket, and prediction tasks inside one system architecture. The prototype uses YOLO-based person detection and tracking, then adds a custom non-maximum-suppression pre-filter and post-frame deduplication layer to reduce duplicate queue entrants. For cases where a person leaves and later returns, the re-entry matcher compares spatial proximity, appearance similarity, and recency before deciding whether to restore an earlier queue number or issue a new one.

The same workflow also produces a printer-agnostic ticket payload with a queue number, short code, QR link, and JWT-backed validation record. The intended deployment prints the payload through a thermal printer, while the demonstration build renders the same data as a PDF ticket. Finally, the dashboard reports the current wait estimate together with 5-, 15-, and 30-minute forecasts. The contribution is therefore the practical integration of detection, queue identity, ticket validation, and prediction for students and administrative staff in an NCF service office setting.

1.5 Theoretical Framework

The theoretical framework of this study is anchored on the queue visibility problem described in the Background of the Study. Campus service offices need a way to observe student arrival, maintain queue order, estimate waiting time, and present useful information to both staff and students. QueueFlow addresses this problem through the combined application of queuing theory, computer vision, predictive analytics, and forecasting theory.

Queuing Theory, as formalized in classical queueing literature [16], explains how arrivals, service capacity, and waiting lines behave through the M/M/c model. The M/M/c model represents a multi-counter service environment in which the arrival rate, service rate, and number of active counters influence queue length, utilization, and waiting time. Together with Little’s Law, this theory provides the mathematical basis for estimating waiting time, queue length, and counter utilization from real-time arrival and service rates. In QueueFlow, queuing theory is connected to the system through queue records, active counter settings, and service-time assumptions.

Computer Vision Theory provides the basis for replacing manual line observation with automated person detection. In QueueFlow, this theory is applied through YOLOv8n and OpenCV, where camera frames are processed to detect persons inside a defined queue zone. YOLO’s single-pass detection approach supports real-time mon-

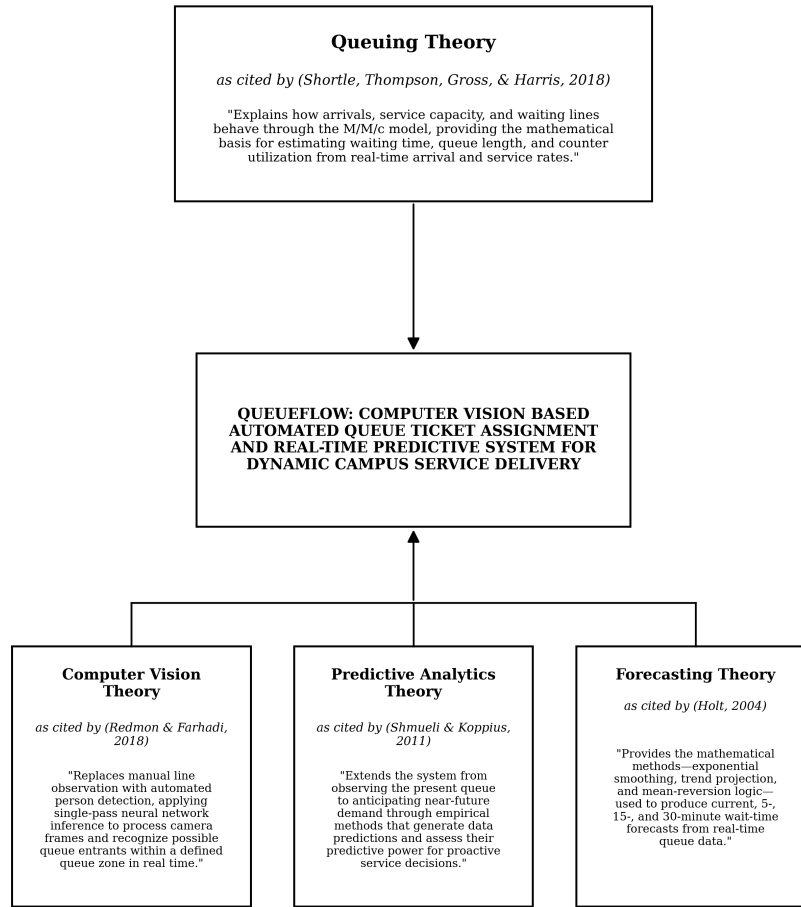


Figure 1: Theoretical Paradigm

Figure 1.1: Theoretical Framework of QueueFlow

itoring because it can identify objects quickly within each frame [3]. Computer vision supplies the first layer of queue visibility: it allows the system to recognize possible queue entrants without requiring every student to manually register at a counter.

Predictive Analytics Theory extends the system from observing the present queue to anticipating near-future demand. As articulated in information systems research, predictive analytics comprises empirical methods that generate data predictions and assess their predictive power for decision support [15]. In QueueFlow, predictive analytics is the conceptual basis for transforming real-time arrival data into proactive staff and counter management.

Forecasting Theory supplies the specific mathematical methods used for the prediction module. QueueFlow applies exponential smoothing, trend projection, and mean-reversion logic—methods rooted in the classical exponentially weighted moving-average framework [17] and treated comprehensively by Hyndman and Athanasopoulos [18]—to

produce current, 5-minute, 15-minute, and 30-minute wait-time estimates. These forecasts support staff decision-making by giving the dashboard more than a static queue count; they provide an early indication of whether congestion is likely to increase or decrease.

These theories divide the QueueFlow problem into workable parts. Computer vision supplies the observable queue activity. Queuing theory relates that activity to service capacity and waiting time. Predictive analytics frames the dashboard as a decision-support tool rather than only a display screen. Forecasting theory supplies the methods used to estimate near-future waiting conditions. The resulting information is shown through the staff dashboard, the student status interface, and the queue ticket workflow. The intended deployment prints tickets on a thermal printer, while the prototype demonstrates the same ticket payload through PDF rendering.

1.6 Conceptual Framework

The conceptual framework uses the Input-Process-Output-Outcome (IPOO) model to show how QueueFlow turns campus service-office conditions into usable queue information. Camera observations, queue-zone settings, counter configuration, staff actions, and service-time assumptions are treated as inputs. The system then processes those inputs through detection, filtering, re-entry matching, ticketing, and prediction before producing the outputs used by staff and students.

Figure 1.2 presents the IPOO-based conceptual framework of the QueueFlow system, showing the relationship among inputs, processes, outputs, outcomes, and feedback.

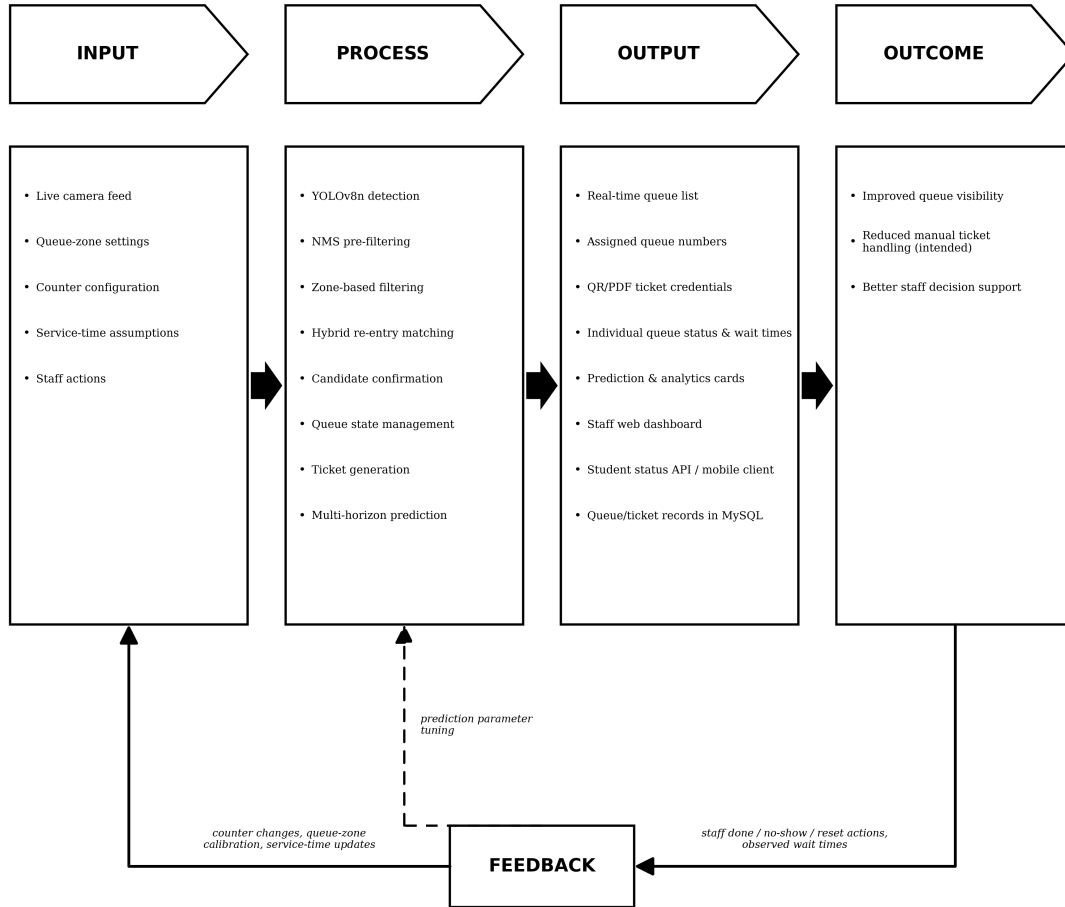


Figure 1.2: Conceptual Framework of QueueFlow

Input. The initial phase consists of the data, settings, and operational conditions required by the system. These include the live camera feed, queue-zone settings, counter configuration, service-time assumptions, staff actions, and the ticket and status-access requirements used by students and administrative staff.

Process. The process stage transforms the input data into queue intelligence through the QueueFlow pipeline. Key processes include YOLOv8n person detection, NMS pre-filtering, zone-based filtering, hybrid re-entry matching, candidate accumulation over a minimum confirmation window, queue state management, no-show detection, position recalculation, ticket generation, and multi-horizon wait-time prediction.

Output. The main outputs are a real-time queue list with assigned queue numbers, ticket credentials rendered through the printer-agnostic protocol (thermal in deployment, PDF in demonstration), individual queue status and wait times, prediction and analytics cards, the staff web dashboard, and the student status API consumed by

a mobile client. Queue entries, ticket records, and operational events are also persisted in the MySQL database.

Outcome. The expected outcome is improved queue visibility and better staff decision support. Reduced manual ticket handling is also an intended outcome of the full thermal-printer deployment.

Feedback. Observed wait times together with staff done, no-show, and reset actions are routed into the Feedback stage. This feedback propagates counter changes, queue-zone calibration, and measured service-time updates back into the Input stage. A secondary signal, shown as a dashed arrow in Figure 1.2, feeds prediction-parameter tuning back into the Process stage, allowing the candidate confirmation window, smoothing factors, and mean-reversion thresholds to be adjusted iteratively. This refinement allows QueueFlow to adapt to the actual demand patterns of NCF service offices rather than remaining locked to its initial settings.

1.7 Statement of the Problem

The study is guided by the following research problems:

1. What are the current limitations of the manual queue management process at NCF service offices during peak demand periods, as established through baseline observation of wait times, ticketing practices, and reported student and staff experiences?
2. How effectively can a YOLO-based passive detection pipeline with hybrid re-identification assign and maintain queue numbers without student interaction, as measured by queue assignment accuracy, re-identification reliability, ticket and token validation, and detection-to-ticket latency?
3. To what degree of accuracy, measured by Mean Absolute Error (MAE) and Mean Absolute Percentage Error (MAPE), can the implemented prediction models forecast current, 5-, 15-, and 30-minute wait times?
4. How do service staff and administrators evaluate QueueFlow across the five dimensions measured by the structured Likert-scale instrument used in this study—namely, usability, detection reliability, prediction usefulness, dashboard clarity, and overall satisfaction—and what overall weighted mean does each dimension obtain?

1.8 Assumptions

The following assumptions delimit the proposed evaluation:

Camera visibility and queue-zone coverage. The camera hardware provides sufficient video quality and coverage of the queue zone to enable reliable YOLO person detection under the lighting conditions and pedestrian densities typical of NCF service areas.

Effectiveness of tracking and re-identification. YOLO’s tracking mechanism, supplemented by the system’s spatial and appearance-based re-identification layer,

produces sufficiently stable person identities across the queue duration to keep duplicate queue assignments to an acceptable rate, to be reported empirically in Chapter 3.

Usefulness of prediction outputs. The implemented prediction models generate forecasts that are actionable and sufficiently accurate for staff scheduling and counter management decisions, with accuracy assessed using MAE and MAPE in Chapter 3.

Ticket rendering. The printer-agnostic ticket protocol functions equivalently whether rendered as a thermal-printed ticket in deployment or as a PDF in the demonstration build, because both render the same queue number, short code, QR link, and JWT.

Staff capability and system adoption. Service staff and administrators possess basic digital literacy to operate the web dashboard and interpret the system's outputs.

1.9 Scope and Delimitation

The scope covers the design, implementation, and evaluation of QueueFlow at NCF in Naga City, Philippines. Included in the prototype are the YOLOv8n detection pipeline, hybrid re-entry matching, multi-horizon prediction, FastAPI REST API, MySQL-backed data layer, staff dashboard, mobile status access, and printer-agnostic ticket generation. The intended deployment renders the ticket through a thermal printer; the current demonstration build renders the same ticket payload as a PDF for evaluation purposes.

The prototype is limited to a single queue zone and one selected service office. It does not currently support simultaneous multi-zone or multi-department management, priority queue differentiation based on student status or urgency factors, direct integration with the Campus Management System or Student Information System, or IoT sensor integration beyond the camera feed and queue ticket printer. The evaluation period is restricted to one semester of post-implementation operation.

1.10 Significance of the Study

QueueFlow is intended to be useful not only as a software prototype but also as a campus-service support tool. Its expected significance is described for the groups that will either operate the system, receive service through it, or use its records for improvement.

Service Staff. Service staff gain a real-time queue dashboard that reduces manual counting and provides data-driven insight into current queue length, individual wait times, and demand forecasts across multiple time horizons.

Students. Students benefit from a passive queueing process that does not require manual registration. Upon entering the monitored area, a queue number is assigned and can be checked through the status link or mobile client using the ticket credential.

Institutional Administrators. Administrators gain queue performance records that support evidence-based decisions about staffing, service hours, and physical layout.

Naga College Foundation, Inc. The institution benefits from a campus-

oriented prototype that demonstrates how automation, computer vision, and predictive analytics can support service delivery and student-facing operations.

Academic Community and Future Researchers. The work provides a local reference for combining computer vision, re-identification, ticket generation, and predictive analytics in Philippine HEI service queues. Future researchers may extend it to multi-camera zones, priority queues, direct campus-system integration, or alternative detection and forecasting models.

1.11 Definition of Terms

Access Token. A ticket credential assigned to each confirmed queue number. In the implemented prototype, this consists of a short code and a JWT-backed validation record that allows the student to check queue status without a login account.

Candidate Accumulation. A buffer mechanism requiring a new detection to appear consistently for a configurable minimum confirmation window before a queue number is assigned, preventing transient detections from generating spurious tickets. The specific window length used in the prototype is tuned empirically and reported in Chapter 3.

Holt’s Double Exponential Smoothing. A classical forecasting algorithm [17] used for the medium-horizon prediction, applying separate smoothing parameters for queue level and trend, together with a damping factor to avoid over-projection. Parameter values used in the prototype are configurable defaults reported and justified in Chapters 2 and 3.

M/M/c Model. A foundational multi-server queueing model from classical queueing theory [16] used for short-term wait-time prediction, calculating $W = L/\lambda$ from Little’s Law using real-time arrival rates from the YOLO detection pipeline.

Mobile Client. The student-facing interface used to check queue status. It is implemented as a mobile-optimized web page consumed through the student status API; references to a “mobile tracker” or “mobile browser” elsewhere refer to the same component.

Non-Maximum Suppression (NMS). Both a standard IoU-based NMS pass and a custom X-column/Y-adjacency filter applied to raw YOLO outputs before they reach the queue tracker, suppressing duplicate head and body boxes YOLO commonly emits for a single person.

Printer-Agnostic Ticket Protocol. The set of fields—queue number, short code, QR link, and JWT—issued for each confirmed queue entry. The same payload can be rendered as a thermal-printed paper ticket (intended deployment) or as a PDF (demonstration build).

Queue Zone. A configurable rectangular region defined by four pixel coordinates on the camera frame; only persons whose bounding box centre falls within this region are tracked and assigned queue numbers.

Re-Entry Matching. A hybrid algorithm that first matches a new detection to a missing person by spatial proximity, then by appearance histogram similarity, and finally by recency as a last resort.

You Only Look Once (YOLO). The YOLOv8n model [4] with `persist=True` tracking, configurable input size, and confidence threshold, used to detect and track persons entering the queue zone in camera frames. Newer variants—YOLOv9 through YOLOv12 [7]—are not used in this study; the rationale is discussed in Section 1.3.

2 Methodology

This chapter explains how QueueFlow will be built, deployed, and evaluated in the NCF service-office setting. It covers the research design, respondents and locale, sampling procedure, data gathering, research instruments, validation of the questionnaire, ethical safeguards, statistical treatment, system architecture, development process, and testing plan.

2.1 Research Design

The study combines developmental work with mixed-methods evaluation. The developmental part covers the actual construction of QueueFlow, while the evaluation part checks whether the prototype performs well enough for a campus service-office setting. Quantitative evidence comes from the system logs, observer records, and survey ratings. These data are used to measure queue assignment accuracy, re-identification reliability, ticket and token validation, detection-to-ticket latency, and wait-time prediction error through Mean Absolute Error (MAE) and Mean Absolute Percentage Error (MAPE). The five-point Likert survey then captures how service staff, administrators, and student users rate dashboard usability, layout clarity, and the usefulness of the prediction outputs after regular exposure to the prototype. Qualitative evidence comes from short semi-structured interviews, which allow respondents to explain operational concerns, useful features, and improvements that are difficult to capture through ratings alone.

The engineering work is organized around the prototype components that carry the main technical risk: YOLOv8n detection, hybrid re-entry matching, ticket-payload generation, multi-horizon prediction, the FastAPI backend, the staff dashboard, and the mobile status interface. Each component is built through the Agile sprint cycle described in Section 2.9 and illustrated in Figure 2.3. Its output is checked against the design principles in Section 2.8. Quantitative results and qualitative accounts are analyzed side by side so that the final discussion can compare measured performance with the actual experience of the users.

2.2 Population and Locale of the Study

The research site is Naga College Foundation, Inc. (NCF) in Naga City, Camarines Sur, Philippines. NCF is an appropriate setting because student transactions in enrollment, cashiering, and library-related services create visible peak periods where manual queue handling is placed under pressure. The prototype will be installed in one service office area chosen with NCF administration. That office serves as the single-queue-zone test site defined in the Scope and Delimitation of Chapter 1.

Three respondent groups are included because they encounter the queue from

different positions. Service staff represent the counter-level users who receive students and process transactions. Administrators and IT personnel represent the group responsible for oversight, deployment, and service-office policy. Currently enrolled NCF students represent the queue participants who receive a ticket and check their status during the evaluation period.

2.3 Sample and Sampling Technique

Sampling is adapted to the size and role of each respondent group rather than applying one procedure to all participants. This allows the evaluation to include the operational perspective of staff, the management perspective of administrators, and the user experience of students.

Service staff. Because the population of staff directly involved in queue-based transactions in the selected service office is relatively small and bounded, total enumeration is used; all eligible staff will be invited to participate. The inclusion criterion is at least two weeks of regular interaction with the system in their normal work role during the evaluation period.

Administrators. Total enumeration is likewise used for this group, since the population of administrators and IT personnel with oversight of the selected service office is small. The inclusion criterion is direct supervisory involvement with the service office during the evaluation period.

Students. Because the student population is large, the sample size for this group is determined using Slovin’s formula at a 5% margin of error:

$$n = \frac{N}{1 + Ne^2} \quad (2.1)$$

where n is the required sample size, N is the population of NCF students who interact with the targeted service office during the evaluation period, and e is the margin of error, set at 0.05. Student respondents will then be drawn through stratified purposive sampling across year levels to ensure that the resulting sample reflects the diversity of users who experience the queue. The inclusion criterion is at least one completed transaction involving a QueueFlow-issued ticket during the evaluation period.

Exclusion criterion. Across all three groups, any prospective respondent below 18 years of age is excluded. The age criterion is enforced through a screening question that immediately precedes the informed consent form, and any prospective respondent indicating an age below 18 is thanked for their interest and not enrolled.

2.4 Data Gathering Procedure

Data gathering is divided into three phases so that the manual baseline, live system behavior, and user feedback are not mixed together prematurely. The sequence also makes it possible to compare what staff normally do, what QueueFlow records during use, and what participants report after using the prototype.

Phase 1: Pre-Deployment Setup and Operational Baseline. Prior to deployment, the researchers will coordinate with the office head of the selected service office to confirm the camera placement, queue zone calibration, counter configuration, and service-time assumptions. During this phase, the researchers will also conduct informal observation of the existing manual queue handling process, recording approximate queue lengths and qualitative accounts from staff regarding the patterns of congestion most commonly encountered. These baseline accounts establish the qualitative reference against which the QueueFlow-mediated process is later compared.

Phase 2: System Deployment and Performance Logging. After alpha and beta testing in Section 2.10, the system will be deployed for regular operation in the selected service office for the evaluation period. During this phase, two parallel streams of data will be captured. First, the system will automatically log every detection event, queue assignment, re-entry matching decision, ticket issuance, prediction output, and dashboard interaction, with associated timestamps. Second, an independent observer will maintain a manual ground-truth log during defined observation windows, recording each actual queue entrant, each genuine re-entry event, and the actual wait time measured from queue zone entry to counter arrival. The automated logs and the manual ground-truth log together support the technical performance metrics specified in Section 2.7.

Phase 3: Post-Evaluation Survey and Interviews. After the evaluation period, the validated questionnaire in Section 2.5 will be administered to the three respondent groups identified in Section 2.3. A subset of respondents from each group will be invited to a short semi-structured interview, scheduled at their convenience and lasting approximately twenty minutes. Interview audio will be recorded only with consent and transcribed, after which the recordings will be deleted. Both survey responses and interview transcripts will be anonymized before analysis.

2.5 Research Instrument

Four instruments provide the evidence for the study: the QueueFlow operational log, the observer’s ground-truth log, a researcher-made Likert questionnaire, and a short semi-structured interview guide.

Operational and ground-truth logs. The operational log is generated automatically by the system and records every detection, queue assignment, re-entry decision, ticket issuance, prediction output, dashboard action, and associated timestamps. The manual ground-truth log is a paper or digital record maintained by an independent observer during defined observation windows in Phase 2. Together these two logs provide the data needed to compute queue assignment accuracy, re-identification reliability, MAE and MAPE for wait-time prediction, detection-to-ticket latency, and ticket and token validation rates.

Likert-scale questionnaire. The questionnaire is structured into five dimensions that correspond directly to Statement of the Problem No. 4 in Chapter 1: usability of the dashboard, mobile client, and ticket workflow; perceived detection reliability of the queue entry process; perceived usefulness of the current, 5-, 15-, and 30-minute wait-time predictions; clarity of the dashboard layout, live snapshot, analytics cards, and counter

controls; and overall satisfaction with the system. Each dimension contains several items rated on the five-point scale shown in Table 2.1.

Semi-structured interview guide. A short interview guide is prepared to explore usability problems, perceived benefits of the prediction outputs, situations in which staff would override system suggestions, and recommendations for future improvement. The guide is used flexibly, with the interviewer free to follow up on respondent comments.

Table 2.1: Likert Scale Interpretation

Scale	Category	WM Range	Interpretation
5	Strongly Agree	4.21–5.00	Strongly Agree / Excellent
4	Agree	3.41–4.20	Agree / Very Satisfactory
3	Neutral	2.61–3.40	Neutral / Satisfactory
2	Disagree	1.81–2.60	Disagree / Poor
1	Strongly Disagree	1.00–1.80	Strongly Disagree / Very Poor

Instrument validation. Because the questionnaire is researcher-made, its content validity and reliability are established before it is used in the main evaluation. Content validation is conducted by three expert raters, namely the thesis adviser, one faculty member with expertise in research methods or statistics, and one faculty member with expertise in computer science or information systems. Each rater independently rates every item on a four-point relevance scale, where 1 means not relevant and 4 means highly relevant. The Content Validity Index is then computed at the item level (I-CVI) and at the scale level (S-CVI/Average). An I-CVI of at least 0.78 per item and an S-CVI/Average of at least 0.90 for the full scale are taken as acceptable thresholds, and any item falling below the threshold is revised or removed based on the experts' written comments before pilot testing.

Reliability testing. After content validation, the questionnaire is pilot-tested with approximately 10 to 15 respondents drawn from the same population strata but excluded from the final sample. Internal-consistency reliability is then computed using Cronbach's alpha, with the formula given in Section 2.7 and a value of at least 0.70 taken as acceptable. If alpha falls below the threshold, items with low item-total correlation are revised and the pilot is repeated. The signed content validation forms and the pilot reliability output are attached as appendices.

2.6 Ethical Considerations

Ethical handling is treated as part of the system design, not only as a research requirement. Staff, administrators, and student users may join surveys, interviews, and system testing only after giving informed consent. Before any questionnaire or interview is administered, the researchers explain the purpose of the study, the activities involved, the expected time commitment, possible benefits, and the participant's right to withdraw without penalty.

Only respondents who are 18 years of age or older will be enrolled. The age criterion is enforced through a screening question that immediately precedes the consent form, and any prospective respondent indicating an age below 18 will not be enrolled. No data from minors is collected or stored at any point in the study.

For confidentiality, personal identifiers are removed before analysis. Questionnaires use respondent codes instead of names. Interview audio, when the respondent allows recording, is transcribed for analysis and then deleted. Results are reported in aggregate form so that individual respondents are not identifiable. Digital files are kept in password-protected storage accessible only to the researchers and the thesis adviser, while hard-copy materials are placed in locked storage and shredded after the retention period. Study data are kept for a maximum of three years after completion and are then destroyed.

Camera Snapshot Handling. The system captures live video frames from the queue zone solely for automated person detection and the live staff dashboard. The detector encodes annotated snapshots, showing the camera frame with queue zone, person bounding boxes, and queue numbers overlaid, at a configurable frame rate and transmits them to the backend through an authenticated endpoint. The backend retains only the latest snapshot in process memory; each new snapshot overwrites the previous one, and no historical snapshot archive is maintained on disk. Where an optional Redis cache is enabled in cloud deployment, it is configured as a volatile cache only: snapshot keys carry a short time-to-live, with a default of ten seconds, and Redis persistence, including RDB snapshotting and AOF append-only logging, is disabled in the bundled Redis configuration (`save ""`; `appendonly no`). If a managed Redis provider is used, persistence and provider-side backups for the QueueFlow cache instance must be disabled, or the existence of provider-side persistence must be disclosed in the deployment notes. Detection metadata, including person tracks, queue length, and wait-time predictions, is transmitted through a separate endpoint and is the only data persisted in the MySQL database; the snapshot image itself is not written to MySQL at any point. The system does not perform facial recognition and does not store biometric templates. Re-identification relies on colour histograms, or clothing-colour distributions derived from bounding-box regions, which are non-biometric and are reset at end-of-session.

Access Control and Transparency. Dashboard access is restricted to authenticated staff through session login, with JWT-based camera-token validation between the detector and the backend. The student-facing mobile client exposes only the requesting student’s own queue status, accessed through the ticket QR code and short-code credential issued to that student; it does not display the camera feed or any other student’s status. Physical signage will be posted at the service-office entrance informing visitors that the area is monitored by an automated queue management system for service-delivery purposes, citing the Data Privacy Act of 2012, or Republic Act No. 10173, and providing researcher contact for data-subject requests. The camera is mounted visibly, not concealed. Consistent with the proportionality principle of the Act, only the minimum data necessary for queue management is processed; no marketing, profiling, or third-party data sharing is performed.

The work is aligned with the Data Privacy Act of 2012, or Republic Act No. 10173, and with applicable NCF research ethics protocols. Ethical clearance will be secured from

the host institution before deployment and data collection begin. Sources are cited and acknowledged throughout the manuscript. Any use of AI-assisted tools is limited to draft organization, language checking, or proofreading; the system implementation, analysis, interpretation, and final responsibility for the manuscript remain with the researchers.

2.7 Statistical Treatment of Data

The statistical treatment is organized around the questions raised in Chapter 1. Technical measures are used for the detector, ticket, and prediction components, while descriptive statistics are used for the respondent ratings and profile data.

Queue Assignment Accuracy and Re-Identification Reliability. These two metrics address Statement of the Problem No. 2. Queue assignment accuracy is computed as the proportion of correctly assigned queue numbers, matching the manual ground-truth log, over the total number of arrivals observed during the evaluation period. Re-identification reliability is computed as the proportion of correct re-entry decisions, either correctly restoring a previous queue number or correctly assigning a new one, over the total number of re-entry events observed:

$$\text{Accuracy} = \frac{\text{correct decisions}}{\text{total decisions}} \times 100\% \quad (2.2)$$

Mean Absolute Error and Mean Absolute Percentage Error. These two metrics address Statement of the Problem No. 3 and are computed separately for the current, 5-minute, 15-minute, and 30-minute prediction horizons:

$$\text{MAE} = \frac{1}{n} \sum |y_{\text{predicted}} - y_{\text{actual}}| \quad (2.3)$$

$$\text{MAPE} = \frac{1}{n} \sum \left| \frac{y_{\text{actual}} - y_{\text{predicted}}}{y_{\text{actual}}} \right| \times 100\% \quad (2.4)$$

where $y_{\text{predicted}}$ is the system's wait-time estimate at a given horizon and y_{actual} is the wait time measured by the observer using a stopwatch from queue zone entry to counter arrival.

Weighted Mean and Overall Weighted Mean. These two measures address Statement of the Problem No. 4. The weighted mean (WM) is computed for each Likert item, each dimension averaged across items within a dimension, and each respondent group: service staff, administrators, and students. The results are then pooled into an overall weighted mean (OWM). Per-group weighted means are reported to allow comparison across stakeholder perspectives.

$$\text{WM} = \frac{\sum(f \times w)}{N} \quad (2.5)$$

where f is the frequency of each response, w is the weight assigned to that response from 1 to 5 on the Likert scale, and N is the number of respondents in the group.

$$\text{OWM} = \frac{\sum \text{WM}}{n} \quad (2.6)$$

where n is the number of items per dimension or the number of dimensions per group, depending on which level of aggregation is being computed. Interpretation follows the verbal categories in Table 2.1.

Percentage Distribution. Percentage is used to describe the composition of the respondent sample by group, year level, role, and demographic profile, and the distribution of responses across the five-point scale for each item:

$$P = \frac{f}{N} \times 100 \quad (2.7)$$

Cronbach’s Alpha. Cronbach’s alpha is used to assess the internal-consistency reliability of the questionnaire after the pilot test:

$$\alpha = \frac{k}{k-1} \left(1 - \frac{\sum \sigma_i^2}{\sigma_{\text{total}}^2} \right) \quad (2.8)$$

where k is the number of items, σ_i^2 is the variance of each item, and σ_{total}^2 is the variance of the total score. Values of 0.70 or higher are taken as acceptable.

Detection-to-Ticket Latency. Detection-to-ticket latency is reported as the mean and standard deviation, in seconds, of the elapsed time from the first confirmed detection of a queue entrant to the issuance of the corresponding ticket payload, computed from the automated operational log.

2.8 System Design and Architecture

QueueFlow is designed as a set of cooperating components rather than as a single monolithic application. The detector, re-identification logic, ticket workflow, prediction module, dashboard, FastAPI backend, and MySQL data layer are developed and tested iteratively so that changes in one component can be checked against the behavior of the rest of the prototype.

2.8.1 Design Principles

Detection Accuracy. The YOLOv8n detection pipeline, supplemented by the custom non-maximum-suppression pre-filter, candidate confirmation window, and hybrid re-entry matching logic, must meet the queue assignment accuracy and re-identification reliability thresholds reported in Chapter 3. These are the primary quality gates before deployment to live operation.

Prediction Usefulness. The prediction module must produce wait-time estimates whose MAE and MAPE, at each of the current, 5-minute, 15-minute, and 30-minute horizons, are low enough to be actionable for staff planning. Specific target thresholds are reported in Chapter 3.

API Reliability. The REST endpoints must handle malformed, empty, or unusually long inputs gracefully, returning structured error responses rather than crashing. Endpoint behavior must be consistent and deterministic across repeated calls with identical inputs, and inference time must remain within bounded latency for live use.

Reproducibility. All model configurations, queue-zone parameters, prediction smoothing factors, and confirmation-window values used in the prototype are recorded as configurable defaults in version-controlled code stored in the project repository, ensuring results reported in this study can be independently verified.

Privacy Protection. Consistent with the visual privacy literature reviewed in Chapter 1, the system processes bounding-box and colour-histogram data rather than storing or transmitting raw video. Personally identifiable images are not retained beyond the live frame buffer required for detection.

2.8.2 Tools and Technologies

The system uses Python as the principal implementation language, with FastAPI as the web framework, OpenCV for camera handling, the Ultralytics implementation of YOLOv8n for detection and persistent tracking, custom Python modules for queue-zone filtering, hybrid re-entry matching, and prediction, and MySQL as the data store. Table 2.2 summarizes the tools and their rationale.

Table 2.2: System Tools and Technologies

Category	Technology	Rationale
Detection	YOLOv8n (Ultralytics)	Lightweight person detection with persistent tracking; rationale for choosing this variant over YOLOv9–v12 is given in Chapter 1.
Backend	Python (FastAPI)	REST API, staff dashboard routes, detector upload endpoints, MJPEG streaming, and student status endpoints.
Video	OpenCV (cv2)	Camera capture, frame preprocessing, annotation, and JPEG encoding for dashboard snapshots.
Tracking	Custom Python	Queue-zone filtering, NMS pre-filter for duplicate head/body box suppression, track-ID remapping, hybrid re-entry matching, and no-show handling.
Prediction	Custom Python	M/M/c wait-time baseline, short trend projection, Holt’s double exponential smoothing, and mean-reversion long forecast.
Ticketing	Thermal printer; demo PDF	Printer-agnostic ticket protocol rendering queue number, short code, QR link, and JWT. Thermal output is the deployment target; PDF rendering is used in the demonstration build.
Database	MySQL	Queue records, short codes, JWTs, no-show events, prediction snapshots, and transaction history.
Auth	Session/JWT	Staff session login, camera-token validation between detector and backend, and student ticket tokens.
Dev Tools	VS Code, Postman, Git/GitHub	Standard development and version control tooling; Postman is used for endpoint contract testing.

2.8.3 System Architecture

The QueueFlow system follows a six-layer architecture composed of the Presentation Layer, the API Layer, the Computer Vision Layer, the Business Logic Layer, the Persistence Layer, and the Cache and Runtime State Layer. This structure reflects the system architecture documented in the project repository and separates user-facing interfaces, API coordination, camera processing, queue operations, database storage, and runtime state management. Figure 2.1 summarizes the layers and their components.

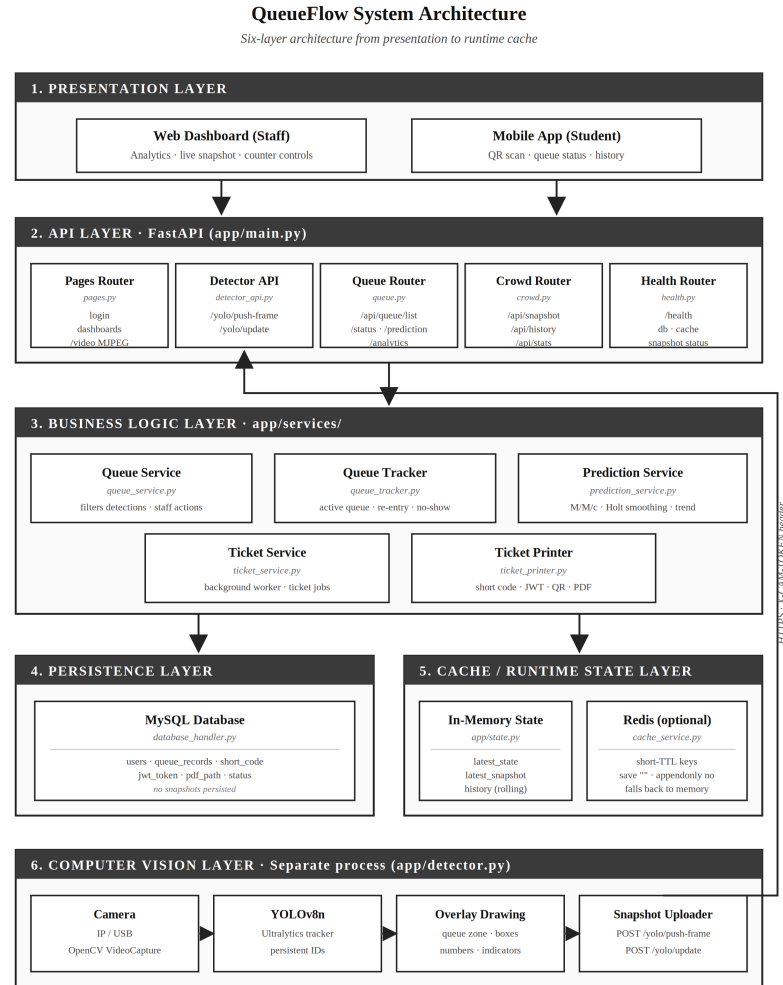


Figure 2.1: QueueFlow six-layer system architecture.
The Computer Vision Layer runs as a separate process and pushes detection data and snapshots up to the API Layer.

Figure 2.1: QueueFlow Six-Layer System Architecture

Presentation Layer. This layer comprises the staff web dashboard, which presents queue analytics, live snapshots or video streaming, counter controls, and no-show alerts, and the student mobile application, which supports QR scanning and queue-status display. These are the only two interfaces exposed to end users.

API Layer. This layer is implemented in Python with FastAPI in `app/main.py`. The backend is composed of five routers: `pages.py` serves the login and dashboard pages and the live MJPEG video stream; `detector_api.py` receives the two endpoints used by the detector process, `POST /yolo/push-frame` for detection metadata and `POST /yolo/update` for annotated snapshots; `queue.py` serves the queue list, queue status, prediction, analytics, and staff queue actions; `crowd.py` serves snapshot retrieval and recent history; and `health.py` reports backend, database, cache, and snapshot status.

Computer Vision Layer. This layer is implemented as a separate Python process in `app/detector.py` running on the camera-connected machine. It opens the camera through OpenCV `VideoCapture`, runs the Ultralytics YOLOv8n model with persistent tracking, draws queue overlays such as queue zone, person boxes, queue numbers, and indicators onto the frame, and uploads the resulting metadata and JPEG snapshot to the backend through the two detector endpoints. Calls are authenticated by a camera token through the `X-CAM-TOKEN` header. Running the detector as a separate process prevents camera processing from blocking dashboard or API requests.

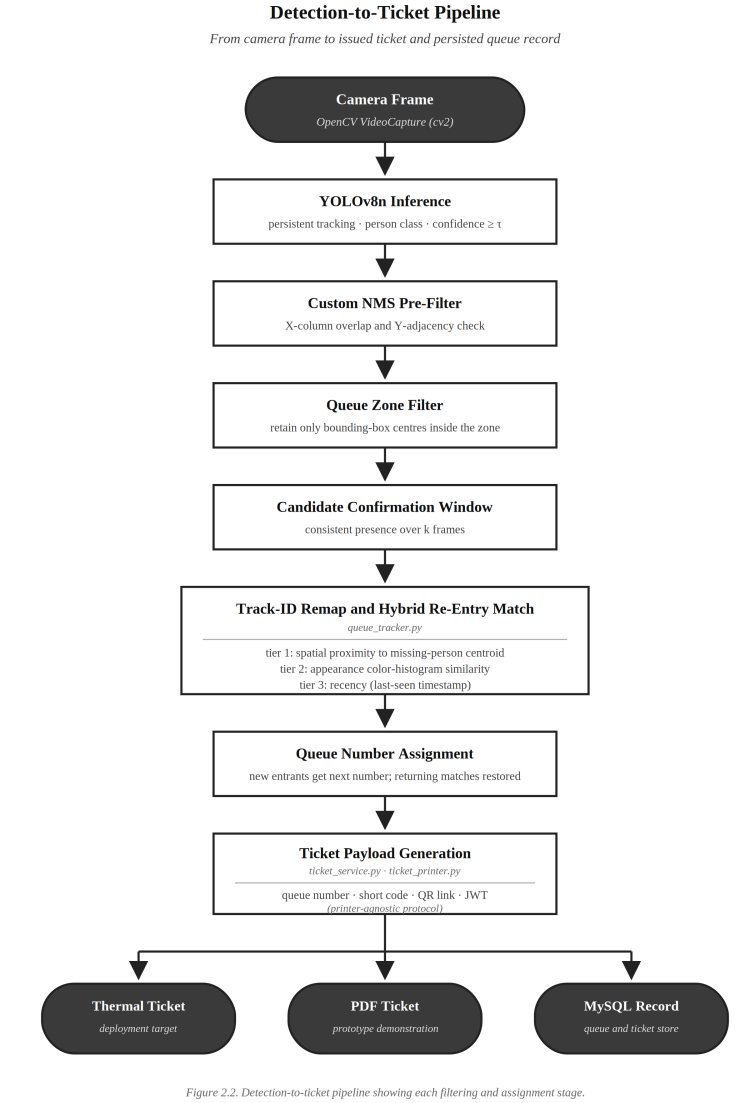


Figure 2.2: QueueFlow Detection-to-Ticket Pipeline

Business Logic Layer. This layer is implemented in `app/services/` and is composed of five services. The Queue Service filters incoming detections, removes weak detections, and routes confirmed detections into the Queue Tracker, which maintains the active queue in memory, assigns queue numbers, handles missing persons and no-show countdowns, and performs hybrid re-entry matching. The Prediction Service computes the current wait-time baseline and the multi-horizon forecasts using the M/M/c queueing approximation, linear trend projection, Holt’s double exponential smoothing, and mean-reversion logic. The Ticket Service is a background worker that consumes ticket-generation jobs, and the Ticket Printer generates the short code, JWT, QR URL, and PDF ticket or, in the deployment configuration, drives the thermal printer.

Persistence Layer. This layer is implemented in `app/database/database_handler.py` against a MySQL database. Persisted data include users, queue records such as queue

number, short code, JWT token, PDF path, status, expiration, creation timestamp, and serving timestamp, and operational events. Raw camera frames, live JPEG snapshots, and the in-memory queue tracker state are intentionally not persisted to MySQL.

Cache and Runtime State Layer. This layer comprises an in-memory state module in `app/state.py` that holds the latest detector state, the latest annotated snapshot, and a rolling history, and an optional Redis cache in `app/services/cache_service.py` that mirrors these values for cloud deployments where multiple backend workers may need to share live data. The Redis cache is configured as volatile only: keys carry short time-to-live values, and Redis persistence, including RDB and AOF, is disabled in the bundled Redis configuration so that snapshots are never written to disk. If a managed Redis provider is used, persistence and backups must be disabled for the QueueFlow cache instance or disclosed in the deployment notes.

2.8.4 Machine-Learning Wait-Time Predictor Training Protocol

QueueFlow does not train its wait-time model while the cashiering prototype is already serving users. Training is handled as an offline step: the researchers prepare the transaction logs, test several predictors on past records, and deploy the selected model only after its error values are acceptable for the study. This keeps experimental model changes away from live queue operations and gives the researchers a reproducible basis for the results presented later in the manuscript.

Dataset. The working dataset will come from historical cashiering transaction logs made available by the NCF Technology Resource Center (TRC), following the data-sharing arrangement described in Section 2.6. The exact date range and final number of usable records will be stated in Chapter 3 after cleaning and screening. The paired BSIS capstone team requested the logs through a formal letter endorsed by the Dean of the College of Computer Studies, and the anonymized records are shared with the BSCS team for QueueFlow predictor development. Expected fields include transaction identifiers, start and end timestamps, transaction type, counter identifier, and computed service duration. Names, student numbers, payment amounts, account details, and other personally identifiable fields are excluded. Only records belonging to the cashiering office selected for deployment are retained for modeling.

Feature Engineering. The predictor uses fields that can be derived from timestamps and from the recent movement of the queue. The initial feature set is organized as follows:

1. Time fields, such as hour, minute, day of week, week of month, and month.
2. Academic-calendar indicators for regular days, enrollment, midterm payment, final payment, and clearance periods, based on the calendar issued by the Office of the Registrar.
3. Queue-state fields, including arrivals and completions within the previous 5, 15, and 30 minutes, estimated queue length, and the number of active counters when this can be inferred from the logs.

4. Lagged wait-time fields, such as the wait time of the previous transaction and recent wait-time averages for comparable hours.

The response variable is transaction wait time. When queue-entry and counter-arrival timestamps are both available, wait time is computed directly from their elapsed time. If the released logs contain service timestamps but not explicit queue-entry timestamps, the reconstruction rule used to estimate waiting time will be documented with the final data-cleaning procedure in Chapter 3.

Train, Validation, and Test Split. Queue records are split by date, not by random row assignment. The first 70% of the ordered records form the training set, the next 15% form the validation set, and the last 15% are reserved for final testing. This order matters because cashiering demand is time-dependent: enrollment week, payment deadlines, and ordinary class days do not behave the same way. A random split could allow future queue patterns to influence model tuning, so it is avoided.

Candidate Model Architectures. The study compares three predictor families:

1. Gradient-boosted regression trees, using XGBoost or LightGBM, because they work well with tabular operational data and can show feature-importance rankings.
2. Random Forest regression, using scikit-learn, because it is a practical comparison model for smaller datasets and is usually less fragile during early tuning.
3. Linear regression, using the same engineered fields, as a baseline for checking whether the tree-based models actually improve prediction error.

The model family with the lowest validation Mean Absolute Error (MAE) at the 15-minute horizon is selected for the prototype. If two models produce nearly identical MAE values, the more stable model across validation periods is preferred. The selected model and its final settings are presented with the results.

Deep sequence models such as Long Short-Term Memory networks and the Temporal Fusion Transformer are not used in this version of the prototype. These architectures may be useful for large, multi-office time-series datasets, but the available cashiering data comes from a single service office. For this study, simpler tabular models are more appropriate and easier to validate.

Hyperparameter Selection. Tuning is limited to a small grid so that the validation set is not overused. For the gradient-boosted models, the grid covers tree depth, learning rate, number of estimators, and L2 regularization, with early stopping based on validation error. For Random Forest regression, tuning is limited to the size of the tree ensemble, the allowed tree depth, and the minimum number of samples required in a leaf node. The final hyperparameters are reported only for the selected model.

Evaluation Protocol. The selected model is evaluated on the held-out test set using MAE and MAPE, following the formulas in Section 2.7. Error is reported separately for the current, 5-minute, 15-minute, and 30-minute horizons. The machine-learning predictor is also compared with the analytical predictor in Section 2.8.3 by running both on the same test records. The comparison reports per-horizon MAE differences and, where the data supports it, a statistical test of whether the observed difference is significant.

Feature-importance results are included to help explain which queue conditions most influence the prediction.

Integration Plan. After offline testing, the trained model is added to the Prediction Service in the Business Logic Layer (Section 2.8.3). For each horizon, the service can produce an analytical estimate, an ML estimate, and a combined estimate. The combined estimate is computed from validation-set performance so that the more reliable component receives greater weight. The dashboard shows the combined value as the default staff-facing estimate, while the component estimates remain available for inspection. The model file is serialized with `joblib`, loaded by the FastAPI backend at startup, and referenced through a configurable path so that future retraining does not require code changes.

Retraining and Model Drift. The procedure in this subsection covers the initial training run only. Periodic retraining, automated drift monitoring, and production retraining pipelines are outside the present scope, although the prototype is arranged so that a later model file can replace the original one.

Current Status. At the time of manuscript preparation, the TRC dataset has been made accessible to the BSCS researchers through the paired BSIS team. Feature engineering and the training script are still being completed. Candidate-model comparison, final error reporting, and live prototype integration will be documented in Chapter 3 once the training and validation run is finished.

2.9 Software Development Process

QueueFlow development is organized as a small Agile-style engineering cycle. Requirements, interface decisions, coding work, testing, and review are repeated across modules instead of being handled as one large build at the end of the study.

Planning and Requirement Analysis. The researchers will collaborate with NCF service office staff to determine system requirements. User needs, expected workflows, dashboard actions, and ticket conventions will be documented to create user stories, prioritize features, and align development goals with the conceptual framework presented in Chapter 1.

System Design. This phase focuses on the architecture and design of the detection pipeline, the backend, the database, and the dashboard. It includes queue-zone coordinate design, ticket-payload schema design, prediction-module parameter design, and dashboard wireframing.

Development Iteration and Sprint Cycles. Development is organized into sprint cycles, each concentrating on a specific module. The sprints, in the order in which they will be executed, are summarized below:

1. YOLOv8n pipeline setup, queue-zone configuration, and the custom NMS pre-filter for duplicate head and body box suppression.
2. Hybrid re-entry matching algorithm design, implementation, and iterative debugging using recorded test footage.

3. No-show detection and queue position recalculation logic.
4. Prediction module implementation of the M/M/c baseline, short-term trend projection, Holt smoothing, and mean-reversion long forecast; iterative accuracy tuning against logged ground-truth wait times.
5. FastAPI backend, staff web dashboard, and counter controls.
6. Printer-agnostic ticket workflow, including JWT issuance, short-code generation, QR linking, and PDF rendering for demonstration.
7. Student status integration, including mobile client, status API, and end-to-end ticket scan tests.
8. Integration testing, performance tuning, and final pre-evaluation hardening.

Figure 2.3 summarizes the per-sprint cycle through which each of the eight sprints listed above is executed. Sprint-level Plan and Design refine the module’s acceptance criteria and interface contracts before Build; Test validates the deliverable against those criteria; and Review captures adviser feedback together with measured performance, which then propagates into the Plan phase of the following sprint. This per-sprint feedback loop instantiates, at the development level, the Feedback stage described in the conceptual framework of Section 1.6.

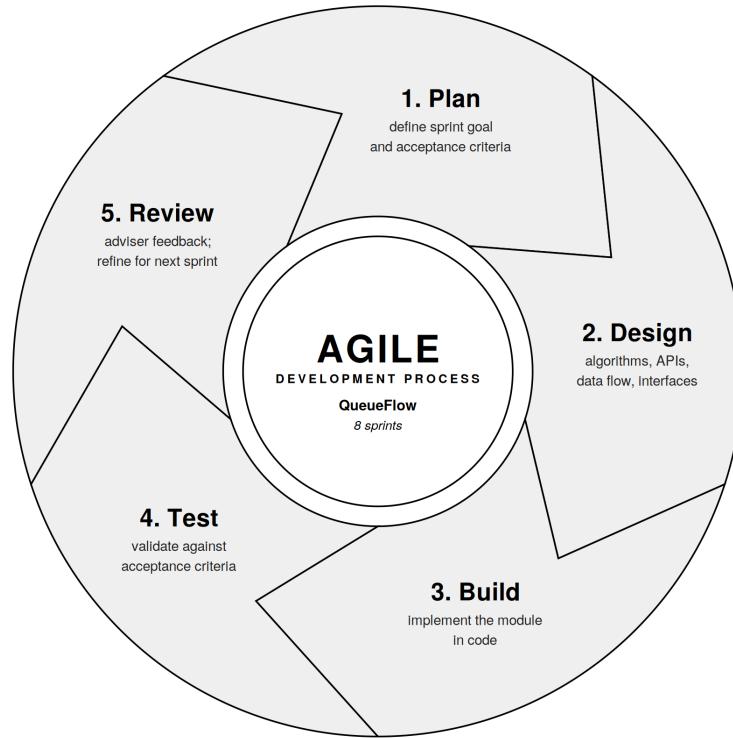


Figure 2.3

Figure 2.3: Agile Development Process for QueueFlow. Each of the eight sprints enumerated above completes one revolution of this five-phase cycle—Plan, Design, Build, Test, Review—before the next sprint begins. The Review phase routes adviser feedback and measured performance back into the Plan phase of the following sprint, instantiating, at the development level, the feedback loop described in Section 1.6.

Each sprint will conclude with a review against defined acceptance criteria and with the thesis adviser before proceeding to the next phase. Sprint reviews will also be used to refine prediction parameters and queue-zone calibration based on observed system behavior, consistent with the feedback loop described in Section 1.6 of Chapter 1.

2.10 Validation and Testing Methods

Validation combines software tests, observed performance, and user feedback. Functional and integration tests check whether the prototype works as intended; queue assignment, re-identification, prediction, and latency measures address the technical research questions; and the survey and interviews address user evaluation.

Alpha Testing. Alpha testing will be conducted internally by the developers using both live camera input and pre-recorded test footage. Defects in detection, tracking, ticket generation, dashboard rendering, and mobile-client status retrieval will be identified

and resolved before any external user is exposed to the system.

Beta Testing. The system will be released to a limited group of NCF service staff for at least two weeks of regular operation. During this period, the developers will observe real-world interaction patterns, collect informal feedback, and log any operational issues. Issues affecting the user experience will be addressed before the structured evaluation begins.

Performance Testing: Queue Assignment Accuracy. During defined observation windows, an independent observer will maintain a manual ground-truth log in which every actual queue entrant will be recorded with a timestamp. The system's queue assignments will then be compared against this log. Accuracy is reported as the proportion of correct assignments over total arrivals, using the formula in Section 2.7.

Performance Testing: Re-Identification Reliability. Re-entry events, in which a tracked person leaves the camera frame and later returns, will also be recorded in the same manual ground-truth log. The system's re-entry decision, either assigning a new number or restoring a previous number, will be compared against the ground-truth identity recorded by the observer. Reliability is reported as the proportion of correct decisions over total re-entry events.

Performance Testing: Prediction MAE and MAPE. Actual wait times will be measured by the observer using a stopwatch from the moment a person enters the queue zone to the moment the person reaches the counter. These observed wait times will be compared to the system's predictions at the corresponding horizons. MAE and MAPE will be computed using the formulas in Section 2.7, separately for the current, 5-minute, 15-minute, and 30-minute horizons.

Performance Testing: Latency and Ticket Flow. Detection-to-ticket latency will be measured from automated timestamps logged at the moment a queue entrant is first detected to the moment the corresponding ticket payload is issued. Ticket flow validation will be performed by issuing tickets across a representative range of conditions and verifying that the short code, QR link, and JWT each correctly resolves to the matching queue status. Failures will be logged and counted toward the ticket and token validation rate reported under Statement of the Problem No. 2.

Usability Evaluation. After a minimum of two weeks of regular system use, the content-validated and pilot-tested questionnaire in Section 2.5 will be administered to the three respondent groups identified in Section 2.3. Responses will be analyzed using the statistical procedures in Section 2.7. A subset of respondents from each group will be invited to a short semi-structured interview, and the resulting transcripts will be thematically coded to surface patterns that the Likert ratings alone do not capture.

The combination of automated performance logs, manual ground-truth observation, structured survey responses, and qualitative interviews provides multiple lines of evidence for each research question, supporting the convergent parallel mixed-methods design described in Section 2.1.

3 Results and Discussions

3.1 Results

3.2 Design and System Plans

3.3 Summary of Findings

3.4 Conclusions

3.5 Recommendations

A **Synthesis Matrix of Related Literature**

Table A.1 presents a synthesis matrix summarizing the reviewed literature, the methodology of each study, key findings, and the gaps identified that motivated the present work.

Table A.1: Synthesis Matrix of Related Literature

Author/s	Year	Topic/Focus	Findings	Gap/Limitation
Sindagi & Patel [8]	2018	CNN-based crowd counting and density estimation	Deep learning enables robust counting under occlusion and scale variation	No queue-number assignment or re-identification
Redmon & Farhadi [3]	2018	YOLOv3 real-time object detection	Single-pass network predicts bounding boxes and class probabilities	No queue-specific deduplication
Jocher et al. [4]	2023	YOLOv8 multi-scale anchor-free detection	Accuracy-speed tradeoff with persistent tracking support	Duplicate head/body boxes in close-range scenarios
Wang et al. [5]	2025	Lightweight YOLOv8n-derived person detector	Supports edge-device surveillance person detection with reduced overhead	Surveillance-focused; no queue ticketing or wait-time prediction
Jocher & Qiu [6]	2024	YOLO11 object detection software	Improves model efficiency and feature reuse in the Ultralytics family	Newer variant; not selected for stable prototype integration
Tian et al. [7]	2025	Attention-centric YOLOv12 detector	Introduces attention-oriented architecture for real-time detection	Benchmark-focused; added complexity for campus prototype hardware
Terven et al. [9]	2023	Comprehensive review of YOLO architectures	Documents innovations from YOLOv1 to YOLOv8 and YOLO-NAS	General review; no queue-specific application
Wojke et al. [11]	2017	DeepSORT online multi-object tracking	Appearance and motion association reduce identity switches during occlusion	No explicit queue-position semantics
Ye et al. [12]	2022	Deep learning for person re-identification	Categorizes feature learning, metric learning, and ranking approaches	No integration with queue management
Wang et al. [13]	2024	Person and vehicle re-identification survey	Reviews recent Re-ID advances and robustness challenges	Broader Re-ID scope; no campus queue workflow
Asperti et al. [14]	2025	Recent person re-identification techniques	Documents newer representation and robustness approaches	Deep Re-ID methods add overhead for a single-zone prototype
Padilla-Lopez et al. [10]	2015	Visual privacy protection methods	Categorizes anonymization, scrambling, and aggregation techniques	Not specific to queue or service-area monitoring
Concepcion et al. [1]	2026	Smart queuing and appointment system in a Philippine HEI	Web-based queue and appointment management can improve client services	User-initiated ticketing; no passive computer-vision arrival detection
Shortle et al. [16]	2018	Foundational queueing theory	M/M/c and Little's Law enable wait-time calculation	No integration with CV inputs
Holt [17]	2004	Exponentially weighted moving average forecasting	Trend and level smoothing capture short-term dynamics	No queue-specific application
Shmueli & Koppius [15]	2011	Predictive analytics as a research paradigm	Empirical prediction supports decision-making	Not applied to live queue systems
Hyndman & Athanasopoulos [18]	2021	Forecasting principles and practice	Treats exponential smoothing and multi-horizon forecasting	Not domain-specific to service queues

B QueueFlow Evaluation Questionnaire

FOR CONTENT VALIDATION – DRAFT

Naga College Foundation, Inc. – College of Computer Studies

Research Title: QueueFlow: Computer Vision Based Automated Queue Ticket Assignment and Real-Time Predictive System for Dynamic Campus Service Delivery

Researchers: Archie D. Balbin and Joshua Rovic P. Sanota

Pre-Screening

Are you 18 years of age or older?

☐ Yes ☐ No

Respondents who answer No are thanked and are not included in the study. No profile answers or survey responses are collected from them.

Section I: Respondent Profile

1. Respondent code assigned by researchers: _____
2. Respondent group: ☐ Service Staff ☐ Administrator / IT Personnel ☐ Student
3. Age range: ☐ 18–22 ☐ 23–30 ☐ 31–40 ☐ 41–50 ☐ 51 and above
4. Gender (optional): _____
5. For students only – Year level: ☐ 1st year ☐ 2nd year ☐ 3rd year ☐ 4th year
☐ 5th year or above
6. For students only – Program / Course: _____
7. For staff and administrators only – Length of service at NCF: ☐ Less than 1 year
☐ 1–3 years ☐ 4–7 years ☐ 8 years or more
8. During the evaluation period, how often did you interact with QueueFlow? ☐ Daily
☐ A few times per week ☐ Weekly ☐ Less than weekly

Instructions

For each statement below, choose the box that best matches your actual experience with QueueFlow during the evaluation period. The items ask about the parts of the

system used by your respondent group. When an item is outside your role, leave it blank or mark “Not Applicable” if that option is provided on the printed form.

Table B.1: Likert Scale Interpretation

Scale	Response	Interpretation
5	Strongly Agree	Strongly Agree / Excellent
4	Agree	Agree / Very Satisfactory
3	Neutral	Neutral / Satisfactory
2	Disagree	Disagree / Poor
1	Strongly Disagree	Strongly Disagree / Very Poor

Tag legend: [S] = Service Staff, [A] = Administrators / IT, [ST] = Students.

Table B.2: Dimension A: Usability

#	Item	Tag	5	4	3	2	1
A1	The QueueFlow dashboard is easy to navigate.	[S] [A]	☐	☐	☐	☐	☐
A2	The actions I need to perform, such as call next, mark done, and mark no-show, are easy to find on the dashboard.	[S]	☐	☐	☐	☐	☐
A3	I can complete my common tasks in QueueFlow without needing help.	[S] [A]	☐	☐	☐	☐	☐
A4	The QueueFlow mobile interface is easy to use on my phone.	[ST]	☐	☐	☐	☐	☐
A5	Scanning the QR code on my queue ticket is straightforward.	[ST]	☐	☐	☐	☐	☐
A6	Checking my queue status on the mobile interface is quick.	[ST]	☐	☐	☐	☐	☐

Table B.3: Dimension B: Detection Reliability

#	Item	Tag	5	4	3	2	1
B1	QueueFlow correctly assigns queue numbers to people who enter the queue area.	[S] [A] [ST]	☐	☐	☐	☐	☐
B2	QueueFlow rarely assigns duplicate queue numbers to the same person.	[S] [A]	☐	☐	☐	☐	☐
B3	When a person briefly leaves the queue area and returns, QueueFlow correctly recognizes them.	[S] [A]	☐	☐	☐	☐	☐
B4	QueueFlow correctly identifies when someone has left the queue without being served.	[S]	☐	☐	☐	☐	☐
B5	I trust the queue list shown on the QueueFlow dashboard.	[S] [A]	☐	☐	☐	☐	☐
B6	The queue number I received matched my actual position in the line.	[ST]	☐	☐	☐	☐	☐

Table B.4: Dimension C: Prediction Usefulness

#	Item	Tag	5	4	3	2	1
C1	The current wait-time estimate matches my actual experience.	[S] [A] [ST]	☐	☐	☐	☐	☐
C2	The 5-minute, 15-minute, and 30-minute wait-time predictions are useful for planning my work.	[S] [A]	☐	☐	☐	☐	☐
C3	The wait-time predictions help me decide when to come back if I cannot wait now.	[ST]	☐	☐	☐	☐	☐
C4	The prediction information is easy to understand.	[S] [A] [ST]	☐	☐	☐	☐	☐
C5	I find the prediction cards on the dashboard useful for managing the queue.	[S] [A]	☐	☐	☐	☐	☐
C6	The predictions are accurate enough to be useful to me.	[S] [A] [ST]	☐	☐	☐	☐	☐

Table B.5: Dimension D: Dashboard Clarity

#	Item	Tag	5	4	3	2	1
D1	The live camera snapshot on the dashboard is clear and easy to interpret.	[S] [A]	☐	☐	☐	☐	☐
D2	The queue list is easy to read at a glance.	[S] [A]	☐	☐	☐	☐	☐
D3	The analytics cards present information in a useful way.	[S] [A]	☐	☐	☐	☐	☐
D4	No-show alerts are clearly visible when they appear.	[S]	☐	☐	☐	☐	☐
D5	The counter controls are clearly labeled.	[S]	☐	☐	☐	☐	☐
D6	The dashboard layout makes it easy to find the information I need.	[S] [A]	☐	☐	☐	☐	☐

Table B.6: Dimension E: Overall Satisfaction

#	Item	Tag	5	4	3	2	1
E1	Overall, QueueFlow improves the queueing experience at the Cashiering Office.	[S] [A] [ST]	☐	☐	☐	☐	☐
E2	QueueFlow makes my work easier.	[S] [A]	☐	☐	☐	☐	☐
E3	QueueFlow makes my transactions less stressful.	[ST]	☐	☐	☐	☐	☐
E4	I would recommend continuing to use QueueFlow after this study.	[S] [A] [ST]	☐	☐	☐	☐	☐
E5	QueueFlow is a meaningful improvement over the previous manual queueing process.	[S] [A] [ST]	☐	☐	☐	☐	☐
E6	I am satisfied with QueueFlow overall.	[S] [A] [ST]	☐	☐	☐	☐	☐

Section VII: Open-Ended Questions

1. What did you like most about using QueueFlow?

2. What did you find most difficult or confusing when using QueueFlow?

3. If you could change one thing about QueueFlow, what would it be?

4. What Cashiering Office situations should QueueFlow handle better before it is considered for wider use?

5. Any other comments or suggestions?

End of Questionnaire. Thank you for your participation.

Questionnaire Notes

The instrument contains 30 Likert-scale statements: six each for Usability, Detection Reliability, Prediction Usefulness, Dashboard Clarity, and Overall Satisfaction. The open-ended questions are placed after the rating items so respondents can explain issues that may not appear in the numerical scores. As described in Sections 2.5 and 2.7, Cronbach's α is computed per dimension after the pilot run.

The tags identify which respondent group should answer each item. This allows the same evaluation form to support staff, administrator / IT, and student feedback while avoiding questions that do not fit a respondent's role. During scoring, dimension means are computed only from the items administered to that respondent group.

Before the questionnaire is used for the main evaluation, it is checked through expert content validation and pilot reliability testing. Items with weak item-total correlation are revised or removed before the final administration. If a Filipino version is needed, translation is done after the English version has passed validation, and the translated items are checked for equivalent meaning.

Bibliography

- [1] A. T. V. Concepcion et al. “Smart Queuing and Appointment Management System for Client Services in Bulacan State University Bustos Campus”. In: *International Journal of Multidisciplinary: Applied Business and Education Research* 7.1 (2026), pp. 5–14. DOI: 10.11594/ijmaber.07.01.02.
- [2] Commission on Higher Education (CHED). *Policies and Guidelines on Outcomes-Based Education and Institutional Sustainability Assessment*. Tech. rep. Quezon City, Philippines: Commission on Higher Education, 2022.
- [3] Joseph Redmon and Ali Farhadi. *YOLOv3: An Incremental Improvement*. 2018. arXiv: 1804.02767.
- [4] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. *Ultralytics YOLOv8*. <https://github.com/ultralytics/ultralytics>. 2023.
- [5] Qiang Wang, Guo Feng, and Zhen Li. “A Lightweight Person Detector for Surveillance Footage Based on YOLOv8n”. In: *Sensors* 25.2 (2025), p. 436. DOI: 10.3390/s25020436.
- [6] Glenn Jocher and Jing Qiu. *Ultralytics YOLO11 (Version 11.0.0)*. <https://github.com/ultralytics/ultralytics>. 2024.
- [7] Yunjie Tian, Qixiang Ye, and David Doermann. *YOLOv12: Attention-Centric Real-Time Object Detectors*. 2025. arXiv: 2502.12524.
- [8] Vishwanath A. Sindagi and Vishal M. Patel. “A Survey of Recent Advances in CNN-based Single Image Crowd Counting and Density Estimation”. In: *Pattern Recognition Letters* 107 (2018), pp. 3–16. DOI: 10.1016/j.patrec.2017.07.007.
- [9] Juan Terven, Diana-Margarita Córdova-Esparza, and Julio-Alejandro Romero-González. “A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS”. In: *Machine Learning and Knowledge Extraction* 5.4 (2023), pp. 1680–1716. DOI: 10.3390/make5040083.
- [10] José Ramón Padilla-López, Alexandros Andre Chaaraoui, and Francisco Flórez-Revue. “Visual privacy protection methods: A survey”. In: *Expert Systems with Applications* 42.9 (2015), pp. 4177–4195. DOI: 10.1016/j.eswa.2015.01.041.
- [11] Nicolai Wojke, Alex Bewley, and Dietrich Paulus. “Simple Online and Realtime Tracking with a Deep Association Metric”. In: *2017 IEEE International Conference on Image Processing (ICIP)*. Beijing, China: IEEE, 2017, pp. 3645–3649. DOI: 10.1109/ICIP.2017.8296962.
- [12] Mang Ye et al. “Deep Learning for Person Re-Identification: A Survey and Outlook”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.6 (2022), pp. 2872–2893. DOI: 10.1109/TPAMI.2021.3054775.

- [13] Z. Wang et al. “A Survey on Person and Vehicle Re-Identification”. In: *IET Computer Vision* 18.8 (2024), pp. 1235–1268. DOI: 10.1049/cvi2.12316.
- [14] Andrea Asperti et al. “A Review of Recent Techniques for Person Re-Identification”. In: *Machine Vision and Applications* 36.1 (2025), pp. 1–29. DOI: 10.1007/s00138-024-01622-3.
- [15] Galit Shmueli and Otto R. Koppius. “Predictive Analytics in Information Systems Research”. In: *MIS Quarterly* 35.3 (2011), pp. 553–572. DOI: 10.2307/23042796.
- [16] John F. Shortle et al. *Fundamentals of Queueing Theory*. 5th ed. Hoboken, NJ: John Wiley & Sons, 2018. ISBN: 978-1-118-94352-6. DOI: 10.1002/9781119453765.
- [17] Charles C. Holt. “Forecasting Seasonals and Trends by Exponentially Weighted Moving Averages”. In: *International Journal of Forecasting* 20.1 (2004), pp. 5–10. DOI: 10.1016/j.ijforecast.2003.09.015.
- [18] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. 3rd ed. Melbourne, Australia: OTexts, 2021.

Curriculum Vitae

Appendices